

CLAUDE CODE DEEP DIVE

Claude Code Subagents (Multi-Agent) — Reference Guide

Source: MULTI_AGENT_GUIDE.md

A self-contained primer on Claude Code **subagents**. Paste this into a new Claude Code session as context whenever you want the model to use, build, or reason about subagents.

■ 1. What is a subagent?

A **subagent** is a specialized AI assistant that Claude Code spawns inside a task to handle a focused job. It has:

- Its **own context window** — separate from the main conversation.
- A **custom system prompt** describing how it should behave.
- An **explicit tool allowlist / denylist** (you decide what it can do).
- Independent **permissions** (modes like `default`, `acceptEdits`, `plan`, `bypassPermissions`).
- Optionally its **own model** (e.g. `haiku` for cheap/fast, `opus` for hard work).

The main agent delegates a task → the subagent works in isolation → only the **summary** comes back to the main conversation. Verbose tool output (greps, log dumps, file reads) stays inside the subagent's context.

Subagents work inside **one session**. If you need agents running across sessions and talking to each other, that's **agent teams**, a separate feature.

■ 2. Why use subagents?

Benefit	What it gets you
Preserve context	Big greps / log dumps / test runs don't pollute the main conversation.
Enforce limits	Restrict tools (e.g. read-only researcher with no <code>Edit</code> / <code>Write</code>).
Reuse	Save user-level subagents in <code>~/.claude/agents/</code> — usable in any repo.
Specialize	Custom system prompts focused on one job (reviewer, debugger, tester).
Control cost	Route routine work to <code>haiku</code> , save <code>opus</code> for the main thread.

■ 3. Built-in subagents

Claude Code ships with a few you don't need to define:

Name	Model	Tools	Used for
Explore	Haiku	Read-only	Fast codebase search / “where is X defined” lookups.
Plan	Inherit	Read-only	Research during plan mode (so plan stays read-only).
general-purpose	Inherit	All	Open-ended multi-step tasks needing both search + edits.
statusline-setup	Sonnet	Read, Edit	When you run <code>/statusline</code> .
Claude Code Guide	Haiku	Bash, Read, Web	Q&A about Claude Code itself.

Claude picks one automatically based on the task. You can also force it (next section).

■ 4. How to invoke a subagent

There are three escalating ways:

a. Natural language — just mention it. Claude *may* delegate.

Use the test-runner subagent to fix failing tests.
Have the code-reviewer look at my recent changes.

TEXT

b. @-mention — *guarantees* this specific subagent runs for one task.

TEXT

```
@"code-reviewer (agent)" look at the auth changes
```

c. **Whole-session subagent** — replace the main thread’s system prompt entirely.

BASH

```
claude --agent code-reviewer
```

...or persist it for the project in `.claude/settings.json`:

JSON

```
{ "agent": "code-reviewer" }
```

■ 5. Defining your own subagent

Easiest path: run `/agents` inside Claude Code and pick **Create new agent** → **Generate with Claude**.
Manual path: drop a Markdown file with YAML frontmatter into one of these locations.

Where to put the file (priority order)

Location	Scope	Priority
Managed settings dir	Org-wide	1 (top)
<code>--agents</code> CLI flag (JSON)	Current session only	2
<code>.claude/agents/</code> (in repo)	This project	3
<code>~/.claude/agents/</code>	All your projects	4
Plugin's <code>agents/</code> directory	Where plugin installed	5

File format

MARKDOWN

```

---
name: code-reviewer
description: Reviews code for quality and best practices. Use proactively after edits.
tools: Read, Glob, Grep
model: sonnet
---
```

You are a senior code reviewer. When invoked:

1. Run `git diff` to see recent changes.
2. Focus on modified files.
3. Flag bugs, security holes, missing tests.

Return: Critical / Warnings / Suggestions, each with a fix snippet.

Frontmatter fields (most useful ones)

Field	Required	Purpose
<code>name</code>	✓	lowercase-hyphenated unique id
<code>description</code>	✓	Tells Claude <i>when</i> to delegate to it
<code>tools</code>		Allowlist of tools (omit = inherit all)
<code>disallowedTools</code>		Denylist (applied before <code>tools</code>)
<code>model</code>		<code>sonnet</code> / <code>opus</code> / <code>haiku</code> / full ID / <code>inherit</code>
<code>permissionMode</code>		<code>default</code> <code>acceptEdits</code> <code>auto</code> <code>dontAsk</code> <code>bypassPermissions</code> <code>plan</code>
<code>mcpServers</code>		Per-subagent MCP servers (inline or by name)
<code>hooks</code>		Lifecycle hooks scoped to this subagent
<code>skills</code>		Skills preloaded into the subagent at start
<code>memory</code>		<code>user</code> / <code>project</code> / <code>local</code> — gives subagent persistent dir
<code>maxTurns</code>		Hard stop after N agentic turns
<code>background</code>		<code>true</code> → always run concurrently with the main session
<code>isolation</code>		<code>worktree</code> → subagent works in a temp git worktree
<code>color</code>		UI color: red/blue/green/yellow/purple/orange/pink/cyan

■ 6. Three ready-to-copy examples

6.1 Code reviewer (read-only)

MARKDOWN

```
----  
name: code-reviewer  
description: Reviews recent diffs for quality, security, perf. Use immediately after  
writing code.  
tools: Read, Grep, Glob, Bash  
model: inherit  
----
```

You are a senior code reviewer.

1. Run `git diff` to see modified files.
2. Check: clarity, naming, duplication, error handling, secrets, input validation, tests.
3. Output 3 buckets – Critical / Warnings / Suggestions – each with a concrete fix.

6.2 Debugger (can edit)

MARKDOWN

```
----  
name: debugger  
description: Debugging specialist for errors and test failures. Use proactively on any  
failure.  
tools: Read, Edit, Bash, Grep, Glob  
----
```

You are an expert debugger.

1. Capture the error + stack.
2. Reproduce locally.
3. Form hypotheses; add temporary logs if needed.
4. Apply the minimal fix and verify.

Return: root cause, evidence, the fix, the test that proves it.

6.3 Read-only DB analyst (with PreToolUse hook)

MARKDOWN

```

---
name: db-reader
description: Run read-only SQL queries to answer data questions.
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---

```

You have read-only DB access. Only `SELECT` queries. If asked to mutate, refuse.

`scripts/validate-readonly-query.sh` reads the JSON hook input from stdin, greps for `INSERT|UPDATE|DELETE|DROP|...`, and `exit 2` to block.

7. Foreground vs background

- **Foreground** (default): main convo blocks until subagent finishes. You see permission prompts.
- **Background**: subagent runs in parallel. Permissions are **pre-approved** before launch; anything not pre-approved is auto-denied at runtime.
- Press **Ctrl+B** mid-task to background a running subagent.
- `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1` disables backgrounding entirely.

8. Common patterns

Pattern	Use when
Isolate noisy output	Running tests, fetching docs, grepping huge logs
Parallel research	"Research auth, db, and api modules in parallel" → 3 subagents at once
Chain subagents	reviewer → optimizer (sequential, each result feeds the next)
Forked subagent	Inherits full conversation; great for "draft tests while I keep coding"

Subagents **cannot spawn other subagents**. For nested delegation, chain from the main thread or use Skills.

■ 9. When NOT to use a subagent

Stay in the main conversation when:

- The task needs frequent back-and-forth.
- Multiple phases share lots of context (plan → implement → test).
- Latency matters — subagents start cold.
- It's a quick targeted edit you'd type in 30 seconds.

For “side question, no tool use needed”, `/btw` is cheaper than spawning a subagent.

■ 10. Cheat-sheet

```
/agents                # interactive create / list / edit
claude agents          # list all configured agents (CLI)
claude --agent code-reviewer # run whole session as that subagent
@"code-reviewer (agent)" ... # invoke once for this turn
Ctrl+B                 # background the current subagent task
```

TEXT

File locations:

```
.claude/agents/<name>.md # project-level (commit it)
~/.claude/agents/<name>.md # user-level (all your projects)
```

TEXT

■ 11. Prompt to feed Claude in the next session

“Read `MULTI_AGENT_GUIDE.md` in the project root. Then create a project-level subagent at `.claude/agents/<name>.md` that **<does X>**. Use **<tools>** only, model **<sonnet/opus/haiku>**, and add a one-line description so Claude knows when to delegate. After creating it, demonstrate it by invoking it on **<concrete task>**.”

Drop that template in, fill the blanks, and you have a reproducible workflow for spinning up new subagents.